OWASP API SECURITY TOP 10 – 1

Dự án Bảo mật Ứng dụng Toàn cầu Mở (OWASP) là một cộng đồng trực tuyến phi lợi nhuận và hợp tác nhằm mục đích cải thiện bảo mật ứng dụng thông qua một bộ nguyên tắc bảo mật, bài viết, tài liệu, v.v. Vào năm 2019, OWASP đã công bố danh sách 10 lỗ hổng API hàng đầu, được thảo luận chi tiết, cùng với tác động tiềm tàng và một vài biện pháp giảm thiểu hiệu quả.

What is an API & Why is it important ?

API là viết tắt của Application Programming Interface (Giao diện Lập trình Ứng dụng). Nó là một phần mềm trung gian giúp tạo điều kiện cho việc giao tiếp giữa hai thành phần phần mềm bằng cách sử dụng một tập hợp các giao thức và định nghĩa. Trong ngữ cảnh API, thuật ngữ 'ứng dụng' đề cập đến bất kỳ phần mềm nào có chức năng cụ thể, và 'giao diện' đề cập đến hợp đồng dịch vụ giữa hai ứng dụng cho phép giao tiếp thông qua các yêu cầu và phản hồi. Tài liệu API chứa tất cả thông tin về cách các nhà phát triển đã cấu trúc các phản hồi và yêu cầu đó. Tầm quan trọng của API đối với việc phát triển ứng dụng được thể hiện chỉ trong một câu duy nhất: API là một khối xây dựng để phát triển các ứng dụng phức tạp và cấp doanh nghiệp.

Recent Data Breaches through APIs

- Vụ rò rỉ dữ liệu LinkedIn: Vào tháng 6 năm 2021, dữ liệu của hơn 700 triệu người dùng LinkedIn đã được rao bán trên một trong những diễn đàn web đen, được đánh cắp bằng cách khai thác API của LinkedIn. Tin tặc đã công bố một mẫu gồm 1 triệu bản ghi để xác nhận tính hợp pháp của vụ rò rỉ LinkedIn, bao gồm tên đầy đủ của người dùng, địa chỉ email, số điện thoại, thông tin định vị địa lý, liên kết hồ sơ LinkedIn, thông tin kinh nghiệm làm việc và các chi tiết tài khoản mạng xã hội khác.
- Vụ rò rỉ dữ liệu Twitter: Vào tháng 6 năm 2022, dữ liệu của hơn 5,4 triệu người dùng Twitter (mở trong tab mới) đã được rao bán trên web đen. Tin tặc đã thực hiện vụ rò rỉ bằng cách khai thác lỗ hổng zero-day trong API của Twitter, cho phép hiển thị tên người dùng Twitter so với số điện thoại di động hoặc email.
- Vụ rò rỉ dữ liệu PIXLR: Vào tháng 1 năm 2021, PIXLR, một ứng dụng chỉnh sửa ảnh trực tuyến, đã bị rò rỉ dữ liệu ảnh hưởng đến khoảng 1,9 triệu người dùng. Tất cả dữ liệu do tin tặc đánh cắp đã được tung lên một diễn đàn web đen, bao gồm tên người dùng, địa chỉ email, quốc gia và mật khẩu đã được mã hóa.

Answer the questions below

In the LinkedIn breach (Jun 2021), how many million records (sample) were posted by a hacker on the dark web?

1 ✓ Correct Answer

Is the API documentation a trivial item and not used after API development (yea/nay)?

nay ✓ Correct Answer

I understand the APIs and am ready to learn OWASP Top 10 Principles.

No answer needed ✓ Correct Answer

1. Vulnerability I – Broken Object Level Authorisation (BOLA)

How does it Happens ?

Nhìn chung, các điểm cuối API được sử dụng cho một hoạt động phổ biến là truy xuất và thao tác dữ liệu thông qua các định danh đối tượng. BOLA đề cập đến Tham chiếu Đối tượng Trực tiếp Không An toàn (IDOR) - tạo ra một kịch bản trong đó người dùng sử dụng chức năng nhập liệu và truy cập vào các tài nguyên mà họ không được phép truy cập. Trong API, các biện pháp kiểm soát như vậy thường được triển khai thông qua lập trình trong các Mô hình (Kiến trúc Mô hình-Xem-Bộ điều khiển) ở level code.

Likely Impact

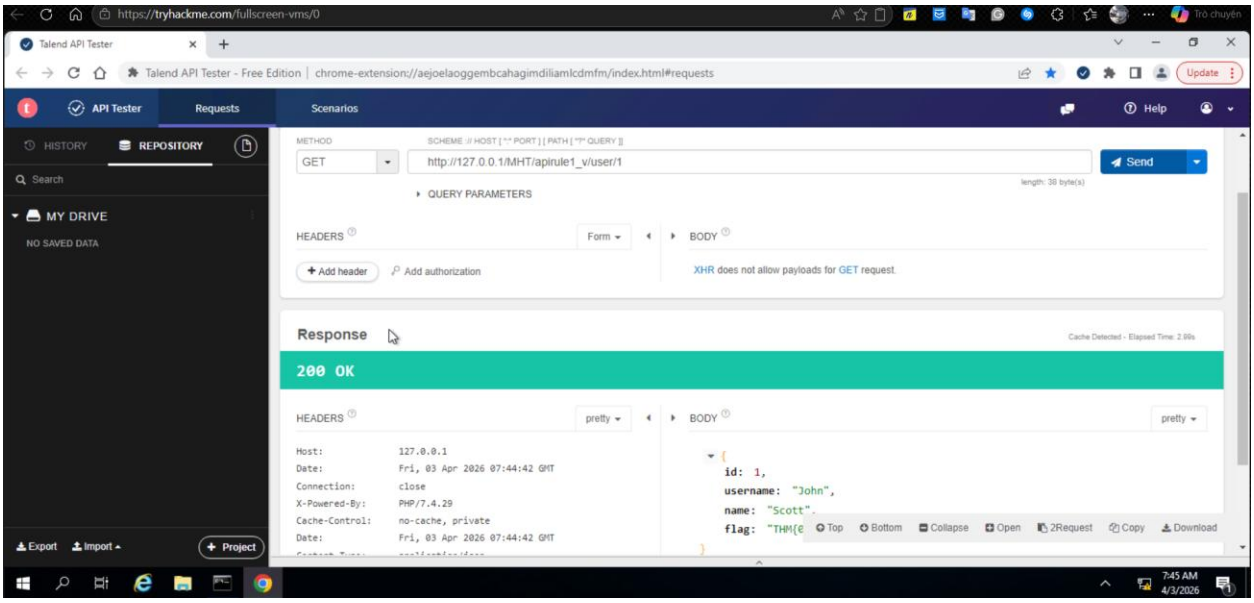
Việc thiếu các biện pháp kiểm soát để ngăn chặn truy cập trái phép vào các đối tượng có thể dẫn đến rò rỉ dữ liệu và, trong một số trường hợp, chiếm đoạt hoàn toàn tài khoản. Dữ liệu người dùng hoặc người đăng ký trong cơ sở dữ liệu đóng vai trò quan trọng đối với uy tín thương hiệu của một tổ chức; nếu dữ liệu đó bị rò rỉ trên internet, điều đó có thể dẫn đến tổn thất tài chính đáng kể.

Practical Example

Ở đây, ta sẽ sử dụng máy ảo được built sẵn để làm theo hướng dẫn và trả lời các câu hỏi.

Mở máy ảo. Chúng ta thấy trình duyệt Chrome và ứng dụng Talend API Tester đang tự động chạy, chúng ta sẽ sử dụng chúng để gỡ lỗi các điểm cuối API.

Bob đang làm việc với tư cách là nhà phát triển API tại Công ty MHT và đã phát triển một điểm cuối /apirule1/users/{ID} cho phép các ứng dụng hoặc nhà phát triển khác yêu cầu thông tin bằng cách gửi ID nhân viên. Trong máy ảo, bạn có thể yêu cầu kết quả bằng cách gửi yêu cầu GET đến http://localhost:80/MHT/apirule1_v/user/1.

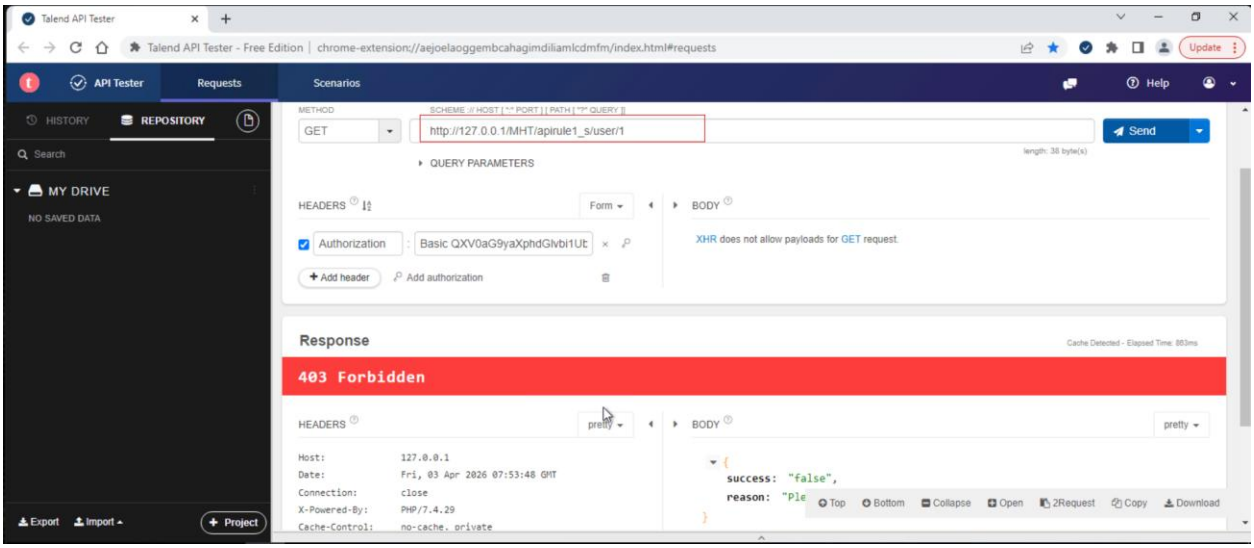


Vấn đề với lệnh gọi API ở trên là gì? Vấn đề là điểm cuối không xác thực bất kỳ lệnh gọi API đến nào để xác nhận xem yêu cầu có hợp lệ hay không. Nó không kiểm tra xem người yêu cầu lệnh gọi API có được phép yêu cầu hay không.

Giải pháp cho vấn đề này khá đơn giản; Bob sẽ triển khai một cơ chế ủy quyền để xác định ai có thể thực hiện các lệnh gọi API để truy cập thông tin ID nhân viên.

Mục đích này đạt được thông qua mã thông báo truy cập hoặc mã thông báo ủy quyền trong tiêu đề. Trong ví dụ trên, Bob sẽ thêm một mã thông báo ủy quyền để chỉ những tiêu đề có mã thông báo ủy quyền hợp lệ mới có thể thực hiện lệnh gọi đến điểm cuối này.

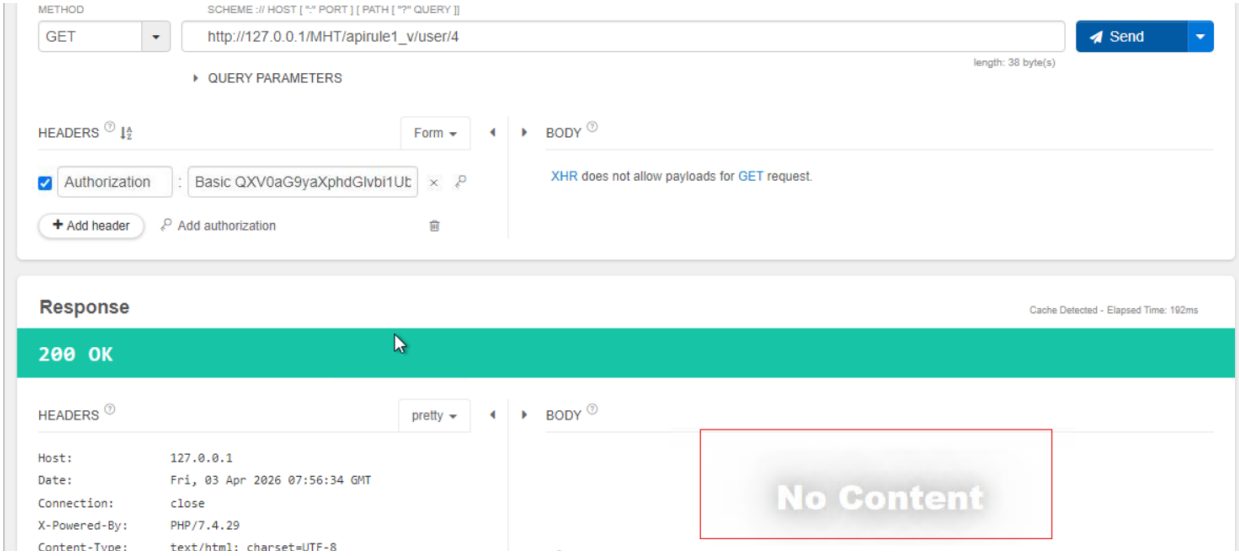
Trong máy ảo, nếu bạn thêm một Authorization-Token hợp lệ và gọi http://localhost:80/MHT/apirule1_s/user/1, thì bạn mới nhận được kết quả chính xác. Hơn nữa, tất cả các lệnh gọi API với mã thông báo không hợp lệ sẽ hiển thị thông báo lỗi 403 Forbidden (như hình bên dưới).



Suppose the employee ID is an integer with incrementing value. Can you check through the vulnerable API endpoint the total number of employees in the company?

_ Check

Ta thử thay đổi ID tới 3, ta vẫn nhận được thông tin trả về của các nhân viên, nhưng tới số 4 thì hết.



Vậy tổng là 3.

Suppose the employee ID is an integer with incrementing value. Can you check through the vulnerable API endpoint the total number of employees in the company?

3 ✓ Correct Answer

Suppose the employee ID is an integer with incrementing value. Can you check through the vulnerable API endpoint the total number of employees in the company?

_ Check

Ta đổi ID thành 2, được flag.

What is the flag associated with employee ID 2?

THM{838123} ✓ Correct Answer

Tương tự, ta đổi ID thành 3, được username cho question 3.

METHOD: GET

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]: http://127.0.0.1/MHT/apirule1_v/user/3

Response

200 OK

HEADERS (pretty)

```
Host:            127.0.0.1
Date:            Fri, 03 Apr 2026 07:59:55 GMT
Connection:      close
X-Powered-By:    PHP/7.4.29
Cache-Control:   no-cache, private
Date:            Fri, 03 Apr 2026 07:59:55 GMT
Content-Type:    application/json
```

▸ COMPLETE REQUEST HEADERS

BODY

```
{
  id: 3,
  username: "Bob",
  name: "Tester",
  flag: "THM{112312}"
}
```

lines nums copy

What is the username of employee ID 3?

Bob ✓ Correct Answer

2. Vulnerability II – Broken User Authentication (BUA)

How does it happen ?

Xác thực người dùng là khía cạnh cốt lõi trong việc phát triển bất kỳ ứng dụng nào chứa dữ liệu nhạy cảm. Lỗi xác thực người dùng (BUA) phản ánh một kịch bản trong đó điểm cuối API cho phép kẻ tấn công truy cập vào cơ sở dữ liệu hoặc giành được đặc quyền cao hơn người dùng hiện có. Nguyên nhân chính dẫn đến BUA là do việc triển khai xác thực không hợp lệ, chẳng hạn như sử dụng truy vấn email/mật khẩu không chính xác, hoặc thiếu các cơ chế bảo mật như tiêu đề ủy quyền, mã thông báo, v.v.

Hãy xem xét một kịch bản trong đó kẻ tấn công có được khả năng lạm dụng API xác thực; điều này cuối cùng sẽ dẫn đến rò rỉ dữ liệu, xóa, sửa đổi hoặc thậm chí là chiếm đoạt hoàn toàn tài khoản bởi kẻ tấn công. Thông thường, tin tặc đã tạo ra các kịch bản đặc biệt để lập hồ sơ, liệt kê người dùng trên hệ thống và xác định các điểm cuối xác thực. Một hệ thống xác thực được triển khai kém có thể dẫn đến việc bất kỳ người dùng nào mạo danh người dùng khác.

Likely Impact

Trong trường hợp xác thực người dùng bị lỗi, kẻ tấn công có thể xâm phạm phiên xác thực hoặc cơ chế xác thực và dễ dàng truy cập dữ liệu nhạy cảm. Những kẻ xấu có thể giả mạo người được ủy quyền và thực hiện các hoạt động không mong muốn, bao gồm cả việc chiếm đoạt hoàn toàn tài khoản.

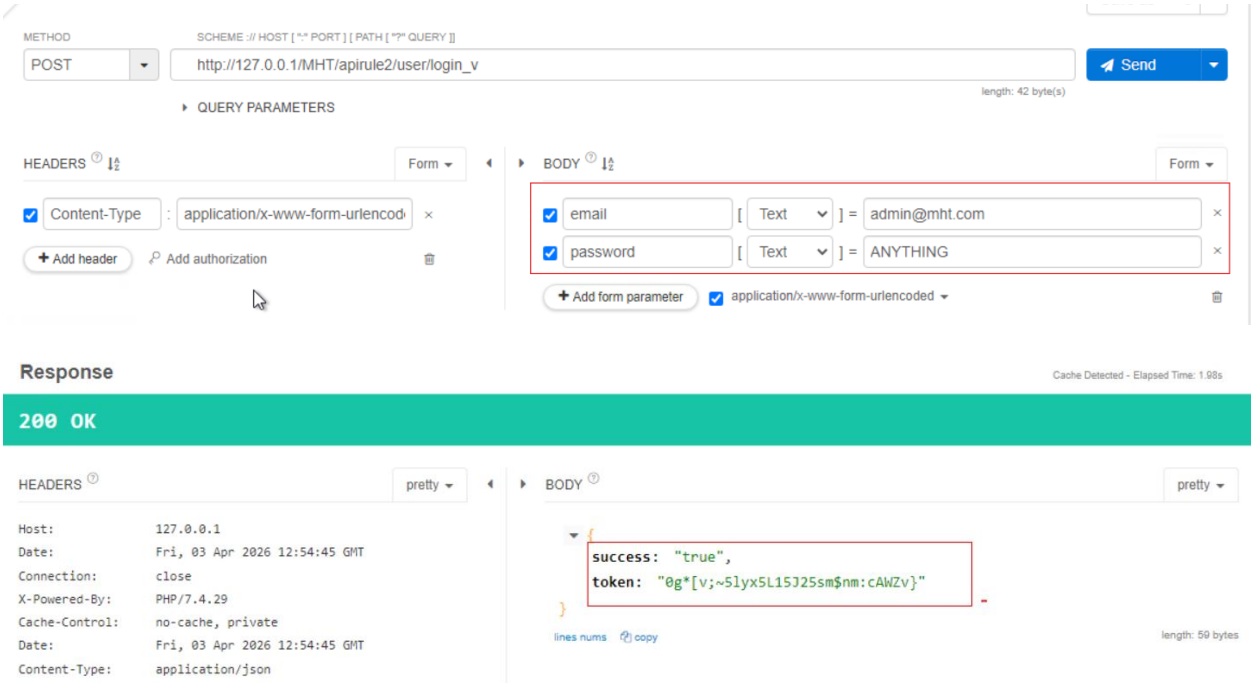
Practical Example

Tiếp tục sử dụng trình duyệt Chrome và Talend API Tester để gỡ lỗi trong máy ảo.

Bob hiểu rằng xác thực là rất quan trọng và được giao nhiệm vụ phát triển một điểm cuối API **apirule2/user/login_v** để xác thực dựa trên email và mật khẩu được cung cấp.

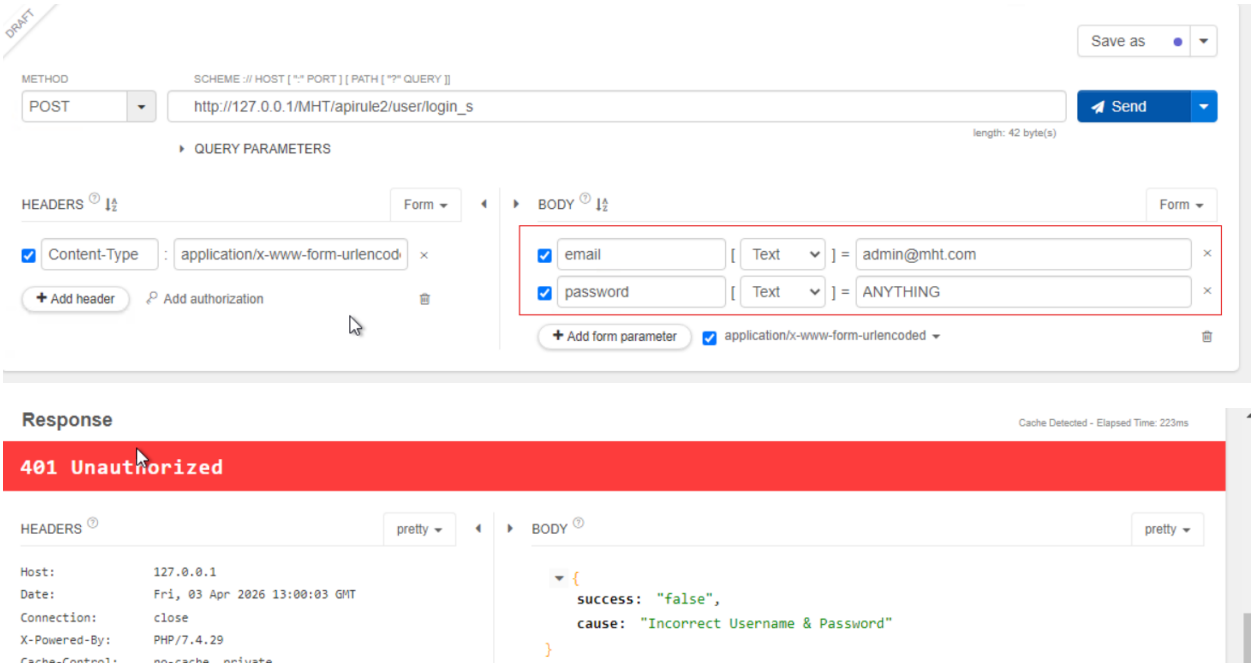
Điểm cuối này sẽ trả về một mã thông báo, mã thông báo này sẽ được chuyển dưới dạng tiêu đề Authorization-Token (yêu cầu GET) đến **apirule2/user/details** để hiển thị chi tiết của nhân viên cụ thể. Bob đã phát triển thành công điểm cuối đăng nhập; tuy nhiên, anh ấy chỉ sử dụng email để xác thực người dùng từ bảng người dùng và bỏ qua trường mật khẩu trong truy vấn SQL. Kẻ tấn công chỉ cần địa chỉ email của nạn nhân để có được mã thông báo hợp lệ hoặc chiếm đoạt tài khoản.

Trong máy ảo, bạn có thể kiểm tra điều này bằng cách gửi yêu cầu POST đến **http://localhost:80/MHT/apirule2/user/login_v** với email và mật khẩu trong các tham số biểu mẫu.



Như chúng ta thấy, điểm cuối dễ bị tấn công đã nhận được một mã thông báo có thể được chuyển tiếp đến /apirule2/user/details để lấy thông tin chi tiết của người dùng.

Để khắc phục điều này, chúng ta sẽ cập nhật logic truy vấn đăng nhập và sử dụng cả email và mật khẩu để xác thực. Điểm cuối /apirule2/user/login_s là một điểm cuối hợp lệ, như được hiển thị bên dưới, cho phép xác thực người dùng dựa trên cả mật khẩu và email.



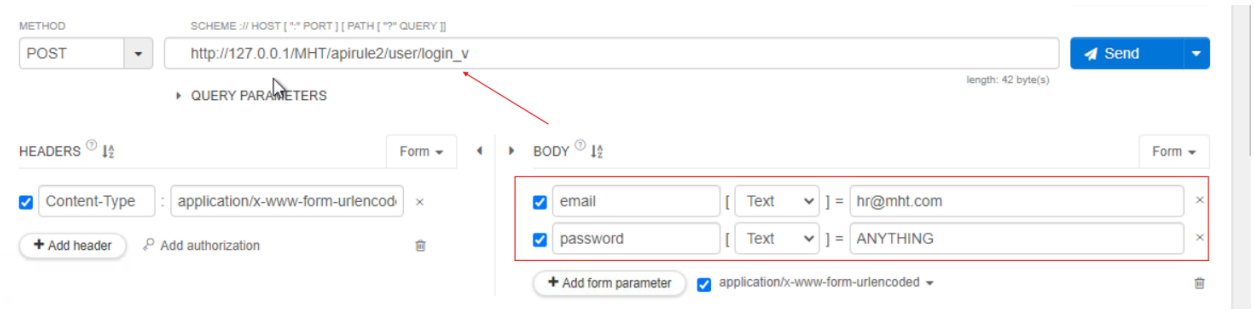
Question 1:

Can you find the token of hr@mht.com?

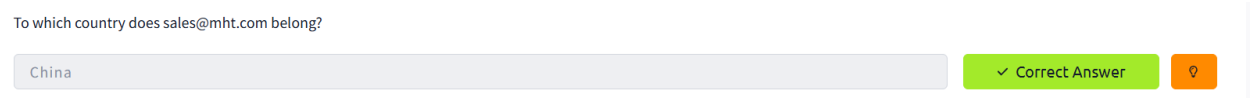
cOC%Aonyis%H)mZ&uJkuI?_W#4&m>Y

✓ Correct Answer

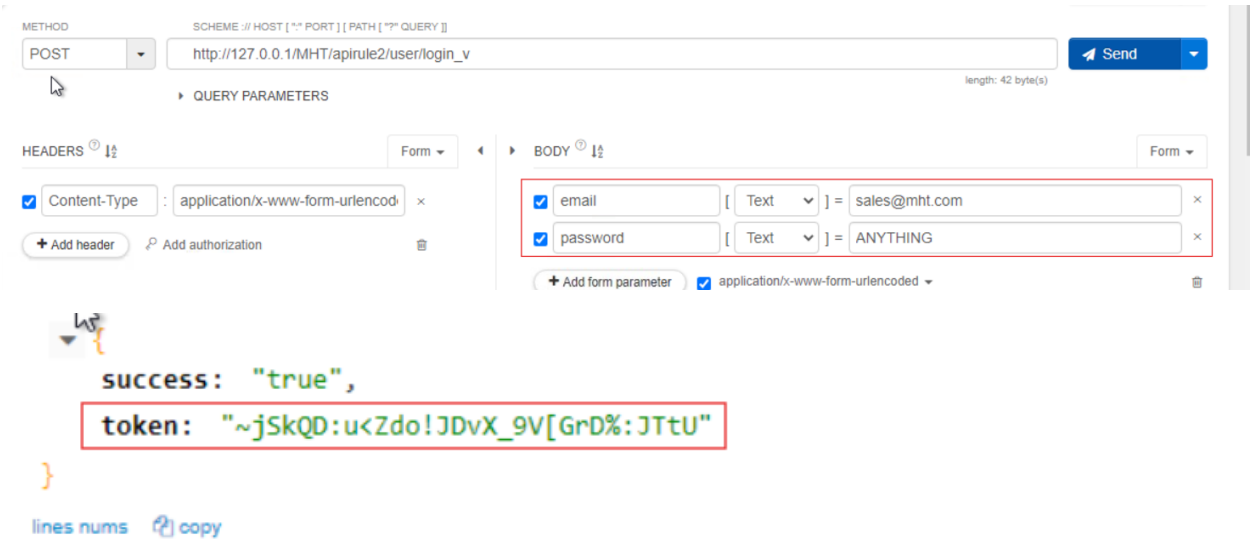
Ở câu hỏi này, ta thay đổi tên email thành hr@mht.com sẽ nhận được flag.



Question 2:

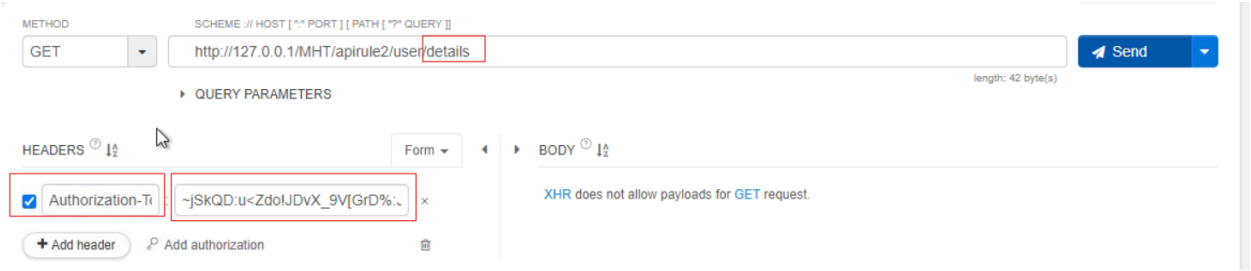


Ta làm tương tự với question 1 để lấy token.



Sau đó, ta đổi sang phương thức GET, ở phần headers, ta “Add header” thêm “Authorization-Token” và nhập value là token ở trên.

Lưu ý, ta đổi URL sang endpoint /apirule2/user/details.



Kết quả

```
{
    id: 3,
    email: "sales@mht.com",
    name: "Joyce",
    token: "~jSkQD:u<Zdo!JDvX_9V[GrD%:JTtU",
    address: "H1 China",
    city: "California",
    country: "China"
}
```


lines nums copy

Question 3:

Is it a good practice to send a username and password in a GET request (yea/nay)?

nay ✓ Correct Answer

3. Vulnerability III – Excessive Data Exposure

How does it Happen ?

Việc lộ dữ liệu quá mức xảy ra khi các ứng dụng có xu hướng tiết lộ nhiều thông tin hơn mức cần thiết cho người dùng thông qua phản hồi API. Các nhà phát triển ứng dụng thường để lộ tất cả các thuộc tính của đối tượng (xét đến các triển khai chung) mà không xem xét mức độ nhạy cảm của chúng. Họ giao nhiệm vụ lọc thông tin cho nhà phát triển giao diện người dùng trước khi hiển thị cho người dùng. Do đó, kẻ tấn công có thể chặn phản hồi thông qua API và nhanh chóng trích xuất dữ liệu bí mật mong muốn. Các công cụ phát hiện trong thời gian chạy hoặc các công cụ quét bảo mật tổng quát có thể đưa ra cảnh báo về loại lỗ hổng này. Tuy nhiên, chúng không thể phân biệt giữa dữ liệu hợp lệ được cho là sẽ được trả về và dữ liệu nhạy cảm.

Likely Impact

Kẻ xấu có thể dễ dàng theo dõi lưu lượng truy cập và truy cập dữ liệu bí mật, bao gồm thông tin cá nhân như số tài khoản, số điện thoại, mã truy cập và nhiều hơn nữa. Thông thường, API sẽ trả về các mã thông báo nhạy cảm, sau này có thể được sử dụng để thực hiện các cuộc gọi đến các điểm cuối quan trọng khác.

Practical Example

Tiếp tục sử dụng trình duyệt Chrome và Talend API Tester để gỡ lỗi trong máy ảo.

Công ty MHT đã ra mắt một cổng thông tin web dựa trên bình luận, thu thập bình luận của người dùng và lưu trữ chúng trong cơ sở dữ liệu cùng với các thông tin khác như vị trí, thông tin thiết bị, v.v., để cải thiện trải nghiệm người dùng.

Bob được giao nhiệm vụ phát triển một điểm cuối (endpoint) để hiển thị bình luận của người dùng trên trang web chính của công ty. Anh ấy đã phát triển một điểm cuối **apirule3/comment_v/{id}** để lấy tất cả thông tin có sẵn cho một bình luận từ cơ sở dữ liệu. Bob cho rằng nhà phát triển giao diện người dùng sẽ lọc thông tin khi hiển thị nó trên trang web chính của công ty.

Vấn đề ở đây là gì? API đang gửi nhiều dữ liệu hơn mức cần thiết. Thay vì dựa vào kỹ sư giao diện người dùng để lọc dữ liệu, chỉ dữ liệu liên quan mới được gửi từ cơ sở dữ liệu.

Bob nhận ra lỗi của mình, đã cập nhật điểm cuối và tạo một điểm cuối hợp lệ **/apirule3/comment_s/{id}** chỉ trả về thông tin cần thiết cho nhà phát triển (như hình bên dưới).

DRAFT

Save as

METHOD: GET

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

http://127.0.0.1/MHT/apirule3/comment_s/1

Send

length: 41 byte(s)

QUERY PARAMETERS

HEADERS — Form

Authorization-Tc : ~jSkQD:u<ZdoIJDvX_9V[GrD%:.

+ Add header — Add authorization

BODY

XHR does not allow payloads for GET request.

200 OK

HEADERS — pretty

Host:	127.0.0.1
Date:	Fri, 03 Apr 2026 13:29:11 GMT
Connection:	close
X-Powered-By:	PHP/7.4.29
Cache-Control:	no-cache, private
Date:	Fri, 03 Apr 2026 13:29:11 GMT
Content-Type:	application/json

BODY

```
{
  postid: "1",
  commenttext: "This is my First Post"
}
```

lines nums — copy

Question 1:

What is the device ID value for post-ID 2?

iOS15.411 — ✓ Correct Answer

Ta đổi endpoint thành **apirule3/comment_v/2**

METHOD: GET

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]

http://127.0.0.1/MHT/apirule3/comment_v/2

Send

length: 41 byte(s)

QUERY PARAMETERS

Response

Cache Det

200 OK

HEADERS — pretty

Host:	127.0.0.1
Date:	Fri, 03 Apr 2026 13:29:59 GMT
Connection:	close
X-Powered-By:	PHP/7.4.29
Cache-Control:	no-cache, private
Date:	Fri, 03 Apr 2026 13:29:59 GMT
Content-Type:	application/json

COMPLETE REQUEST HEADERS — pretty

BODY

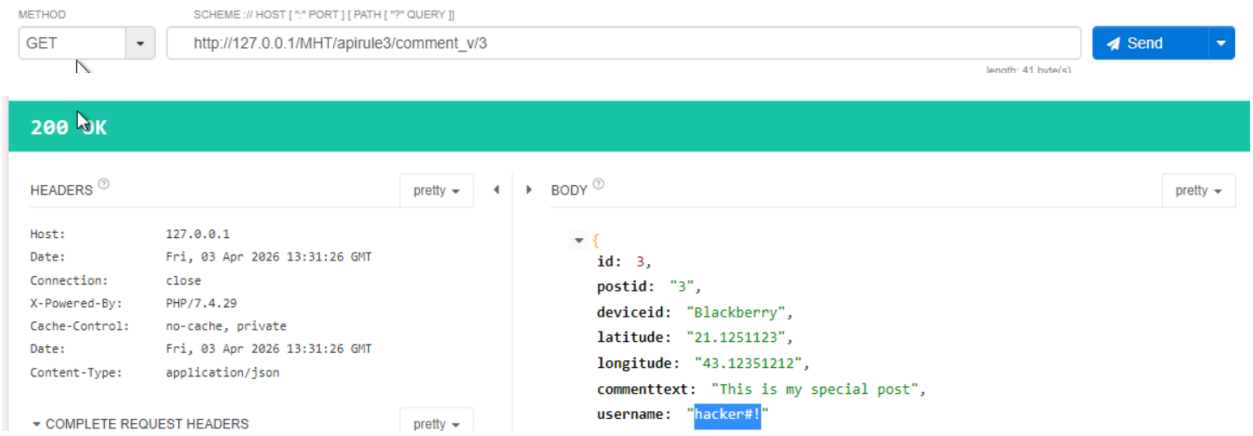
```
{
  id: 2,
  postid: "2",
  deviceid: "iOS15.411",
  latitude: "34.12312311",
  longitude: "54.123123123",
  commenttext: "This is another Post on the Blog.",
  username: "anotheruser007"
```

Question 2:

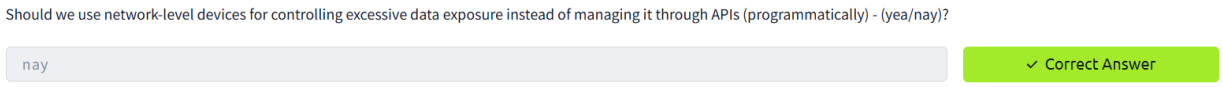
What is the username value for post-ID 3?

hacker#! — ✓ Correct Answer

Tương tự như question 1



Question 3:



Giải thích: Network-level hữu ích cho lớp bảo vệ ngoài (perimeter), nhưng không thể thay thế việc quản lý excessive data exposure programmatically ở tầng API/application. Nếu chỉ dựa vào network, bạn vẫn dễ bị exploit vì root cause nằm ở code.

4. Vulnerability IV – Lack of Resources & Rate Limiting

How does it Happen ?

Việc thiếu tài nguyên và giới hạn tốc độ truy cập đồng nghĩa với việc các API không thực thi bất kỳ hạn chế nào về tần suất yêu cầu tài nguyên của máy khách hoặc kích thước tệp, điều này ảnh hưởng nghiêm trọng đến hiệu suất máy chủ API và dẫn đến tấn công từ chối dịch vụ (DoS) hoặc không khả dụng dịch vụ. Hãy xem xét một kịch bản trong đó giới hạn API không được thực thi, cho phép người dùng (thường là kẻ xâm nhập) tải lên nhiều tệp có dung lượng vài GB cùng một lúc hoặc thực hiện bất kỳ số lượng yêu cầu nào mỗi giây. Các điểm cuối API như vậy sẽ dẫn đến việc sử dụng tài nguyên quá mức trong mạng, lưu trữ, tính toán, v.v.

Likely Impact

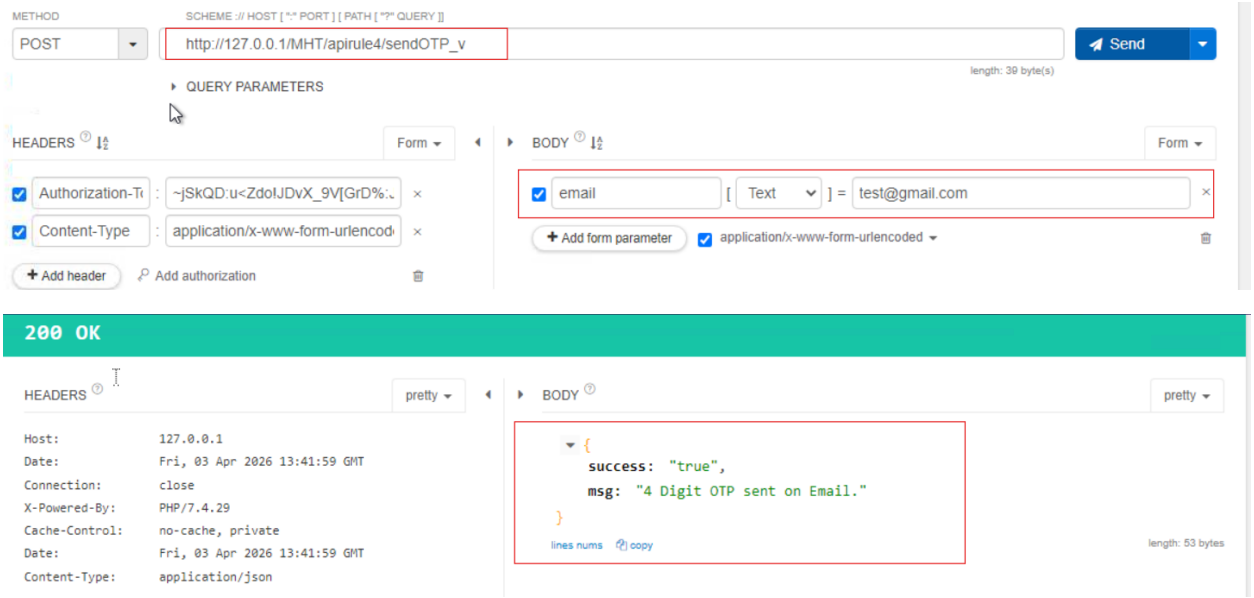
Cuộc tấn công chủ yếu nhắm vào các nguyên tắc về tính khả dụng của bảo mật; tuy nhiên, nó có thể làm tổn hại đến danh tiếng thương hiệu và gây thiệt hại về tài chính.

Practical Example

Tiếp tục sử dụng trình duyệt Chrome và Talend API Tester để gỡ lỗi trong máy ảo.

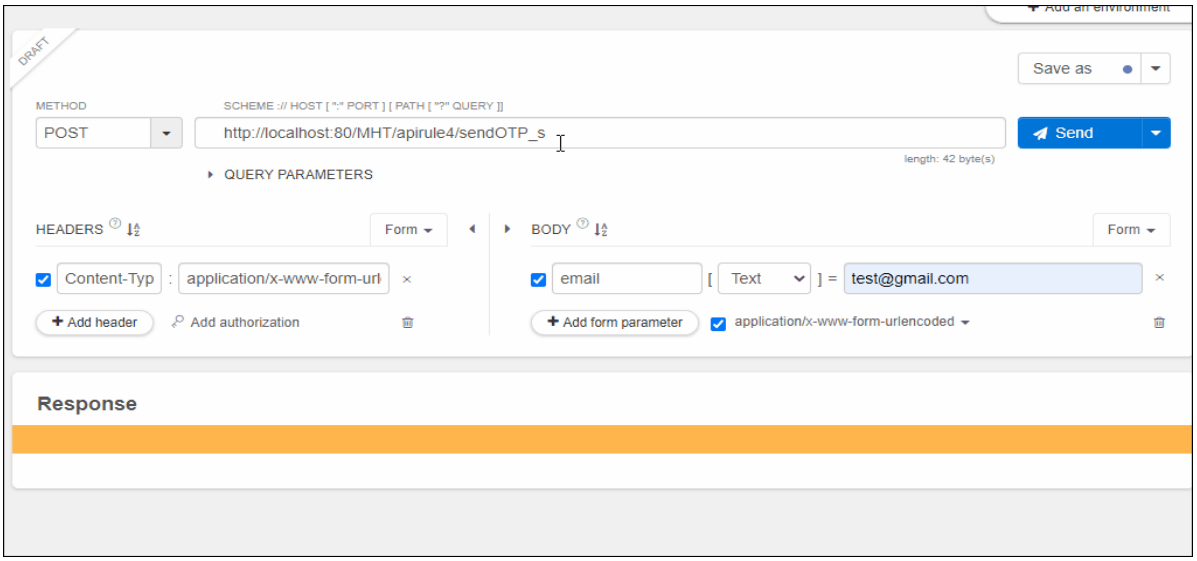
Công ty MHT đã mua một gói tiếp thị qua email (20.000 email mỗi tháng) để gửi email tiếp thị, email khôi phục mật khẩu, v.v. Bob nhận ra rằng anh ấy đã phát triển thành công API đăng nhập, nhưng cần phải có tùy chọn "Quên mật khẩu" có thể được sử dụng để khôi phục tài khoản.

Anh ấy bắt đầu xây dựng một điểm cuối **/apirule4/sendOTP_v** sẽ gửi mã số gồm 4 chữ số đến địa chỉ email của người dùng. Người dùng đã được xác thực sẽ sử dụng Mật khẩu một lần (OTP) đó để khôi phục tài khoản.



Vấn đề ở đây là gì? Bob chưa bật bất kỳ tính năng giới hạn tốc độ nào trên điểm cuối. Kẻ xấu có thể viết một đoạn mã nhỏ và tấn công Brute-force điểm cuối, gửi nhiều email trong vài giây và sử dụng gói tiếp thị email mà công ty mới mua (gây thiệt hại về tài chính).

Cuối cùng, Bob đã nghĩ ra một giải pháp thông minh (/apirule4/sendOTP_s) và bật tính năng giới hạn tốc độ sao cho người dùng phải đợi 2 phút để yêu cầu mã OTP một lần nữa.



Question 1:

Can rate limiting be carried out at the network level through firewall etc. (yea/nay)?

yea

✓ Correct Answer

Yea — bạn có thể (và nên) dùng firewall/WAF để rate limiting ở network level, nhưng không nên coi đó là giải pháp duy nhất hoặc chính. Đối với API, rate limiting hiệu quả nhất vẫn cần application-level awareness để tránh bypass và bảo vệ đúng business logic.

Question 2:

What is the HTTP response code when you send a POST request to **/apirule4/sendOTP_s** using the email address hr@mht.com?

200

✓ Correct Answer

METHOD: POST

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]: http://127.0.0.1/MHT/apirule4/sendOTP_s

Send

length: 39 byte(s)

QUERY PARAMETERS

HEADERS — Form

Content-Type : application/x-www-form-urlencod

+ Add header | Add authorization

BODY — Form

email [Text] = hr@mht.com

+ Add form parameter | application/x-www-form-urlencoded

Response

Cache Detected - Elapsed Time: 266ms

200 OK

Question 3:

What is the "msg key" value after an HTTP POST request to **/apirule4/sendOTP_s** using the email address sale@mht.com?

Invalid Email

✓ Correct Answer

METHOD: POST

SCHEME :// HOST [":" PORT] [PATH ["?" QUERY]]: http://127.0.0.1/MHT/apirule4/sendOTP_s

Send

length: 39 byte(s)

QUERY PARAMETERS

HEADERS — Form

Content-Type : application/x-www-form-urlencod

+ Add header | Add authorization

BODY — Form

email [Text] = sale@mht.com

+ Add form parameter | application/x-www-form-urlencoded

200 OK

HEADERS — pretty

Host:	127.0.0.1
Date:	Fri, 03 Apr 2026 13:48:49 GMT
Connection:	close
X-Powered-By:	PHP/7.4.29
Cache-Control:	no-cache, private
Date:	Fri, 03 Apr 2026 13:48:49 GMT
Content-Type:	application/json

BODY — pretty

```
{
    success: "false",
    msg: "Invalid Email"
}
```

lines nums | copy

length: 41 bytes

5. Vulnerability V – Broken Function Level Authorisation

How does it Happen ?

Lỗi ủy quyền cấp chức năng phản ánh kịch bản trong đó người dùng có đặc quyền thấp (ví dụ: nhân viên bán hàng) vượt qua các bước kiểm tra hệ thống và truy cập dữ liệu bí mật bằng cách mạo danh người dùng có đặc quyền cao (Quản trị viên). Hãy xem xét kịch bản về các chính sách kiểm soát truy cập phức tạp với nhiều cấp bậc, vai trò và nhóm khác nhau, cùng với sự phân tách không rõ ràng giữa các chức năng thông thường và chức năng quản trị, dẫn đến các lỗ hổng ủy quyền nghiêm trọng. Bằng cách lợi dụng những vấn đề này, kẻ xâm nhập có thể dễ dàng truy cập vào các tài nguyên trái phép của người dùng khác hoặc, nguy hiểm nhất – các chức năng quản trị.

Lỗi ủy quyền cấp chức năng phản ánh quyền IDOR, trong đó người dùng, rất có thể là kẻ xâm nhập, có thể thực hiện các tác vụ cấp quản trị. Các API với vai trò và quyền người dùng phức tạp có thể trải rộng trên nhiều cấp bậc dễ bị tấn công kiểu này hơn.

Likely Impact

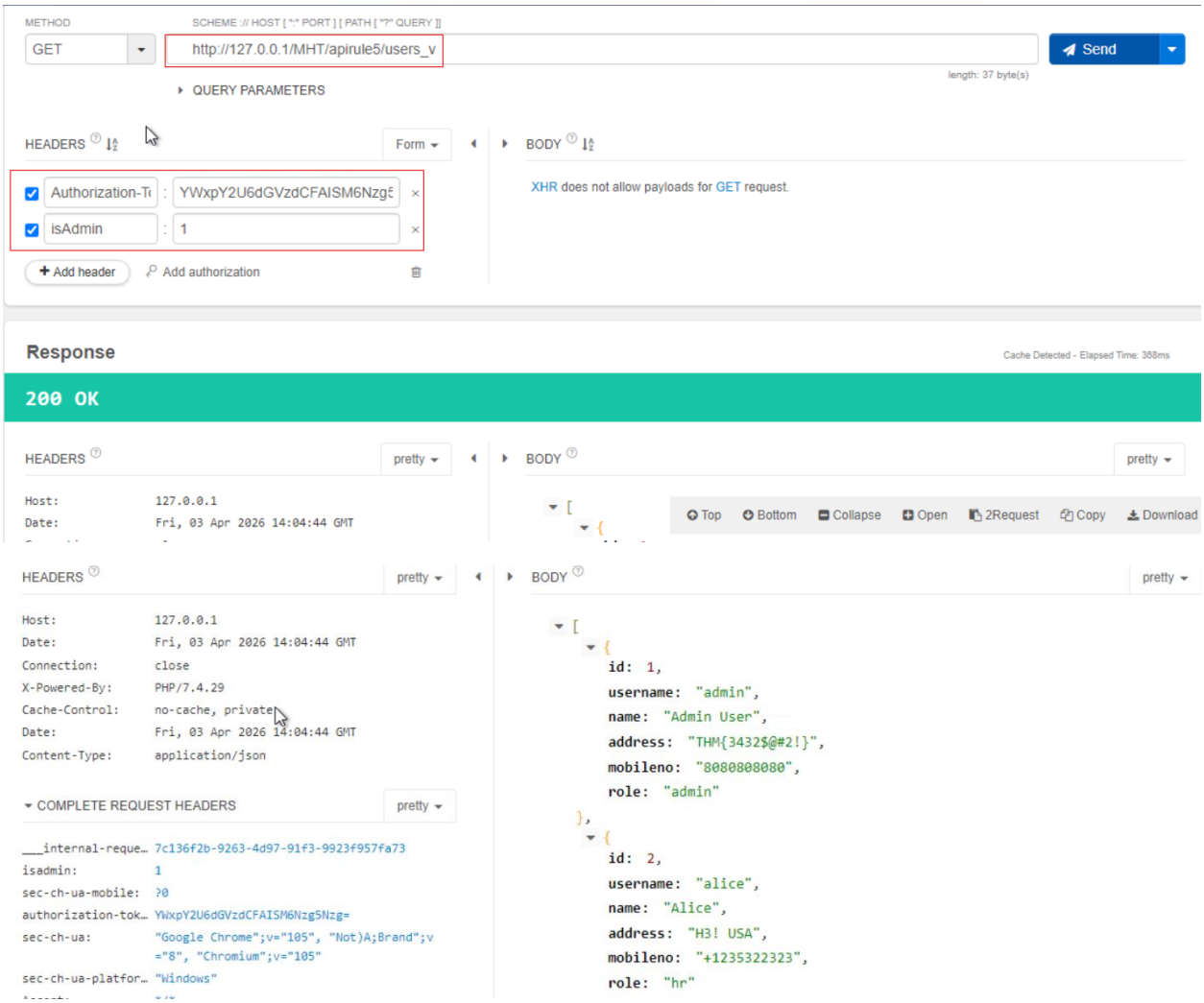
Cuộc tấn công chủ yếu nhắm vào các nguyên tắc ủy quyền và không thể phủ nhận trong bảo mật. Việc phá vỡ ủy quyền cấp độ chức năng có thể cho phép kẻ xâm nhập mạo danh người dùng được ủy quyền và giành được quyền quản trị để thực hiện các tác vụ nhạy cảm.

Practical Example

Tiếp tục sử dụng trình duyệt Chrome và Talend API Tester để gỡ lỗi trong máy ảo.

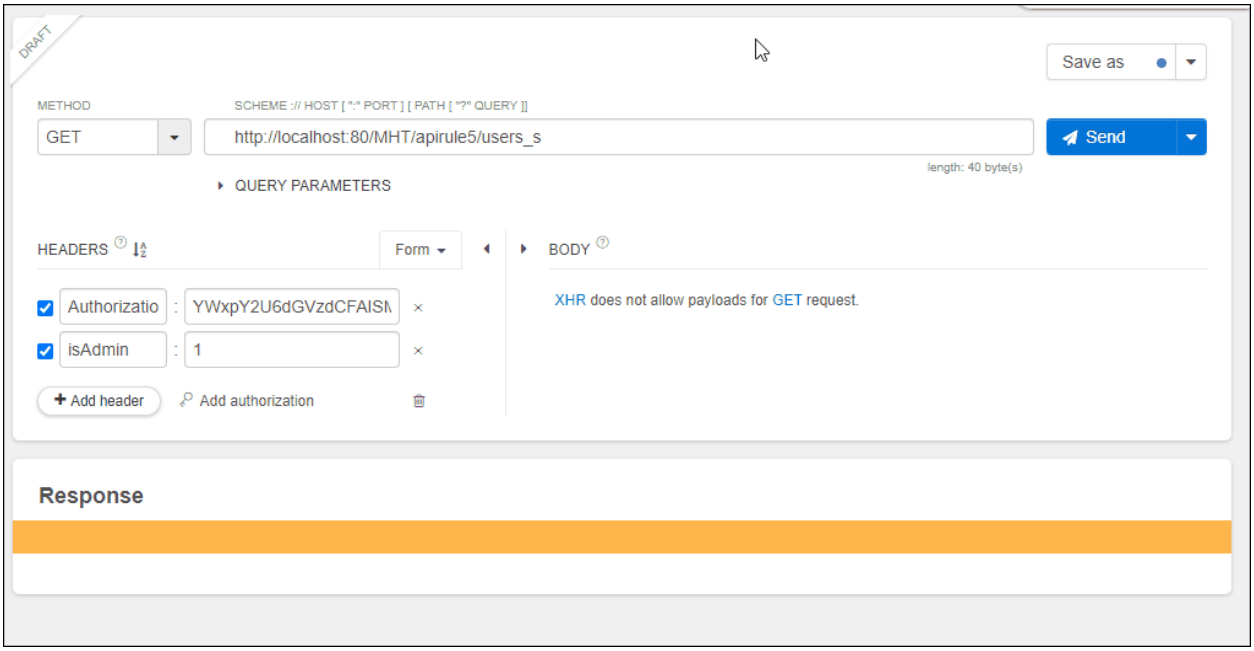
Bob được giao thêm một nhiệm vụ nữa là phát triển bảng điều khiển quản trị cho các giám đốc điều hành công ty để họ có thể xem tất cả dữ liệu của nhân viên và thực hiện các tác vụ cụ thể.

Bob đã phát triển một điểm cuối /apirule5/users_v để lấy dữ liệu của tất cả nhân viên từ cơ sở dữ liệu. Để tăng cường bảo mật, anh ấy đã thêm một lớp bảo mật khác bằng cách thêm tiêu đề đặc biệt isAdmin vào mỗi yêu cầu. API chỉ lấy thông tin nhân viên từ cơ sở dữ liệu nếu isAdmin=1 và Authorization-Token chính xác. Mã thông báo ủy quyền cho người dùng HR Alice là YWxpY2U6dGVzdCFAISM6Nzg5Nzg=.



Chúng ta có thể thấy rằng Alice là người dùng không phải quản trị viên (HR) nhưng có thể xem tất cả dữ liệu của nhân viên bằng cách thiết lập các yêu cầu tùy chỉnh đến điểm cuối với giá trị isAdmin = 1.

Vấn đề này có thể được giải quyết bằng lập trình bằng cách triển khai các quy tắc ủy quyền chính xác và kiểm tra vai trò chức năng của mỗi người dùng trong cơ sở dữ liệu trong quá trình truy vấn. Bob đã triển khai một điểm cuối khác /apirule5/users_s để xác thực vai trò của mỗi người dùng và chỉ hiển thị dữ liệu của nhân viên nếu vai trò là Quản trị viên.



Question 1:

Sử dụng thông tin ở trên

What is the mobile number for the username Alice?

+1235322323 ✓ Correct Answer

Question 2:

Is it a good practice to send isAdmin value through the hidden fields in form requests - yea/nay?

nay ✓ Correct Answer

Hidden fields chỉ nên dùng cho dữ liệu không nhạy cảm và vẫn phải validate nghiêm ngặt ở server (ví dụ: ID của object đang chỉnh sửa, token CSRF...).

Question 3:

Sử dụng thông tin ở trên

What is the address flag of username admin?

THM{3432$@#2!} ✓ Correct Answer